

From Dirty Data to Predictive Models

Titanic survival prediction with Naive Bayes and Linear Regression

Author: Mit Sheth | ms7556

Colab notebook: [assign1-ms7556.ipynb](#)

Page map

Part of the assignment	Report section	Pages
Problem definition and dataset	1. Introduction	2-3
Step 1 - Data Cleaning	2. Data Cleaning	3-4
Step 2 - Feature Engineering	3. Feature Engineering	4-6
Step 3 - Model Training; Step 4 - Model Evaluation	4. Model Comparison	6-10
Interpretation, strengths and limitations	5. Discussion	10-11
AI Tool Usage Disclosure	6. AI Tool Usage Disclosure	11-12

Summary

We predict whether a Titanic passenger survived from 891 labeled records. The raw data contains missing values (**Age** 19.87%, **Cabin** 77.10%, **Embarked** 0.22%) and text fields that models cannot use directly. We clean the data without leaking information from the test set, engineer features and compare a generative model (Bernoulli Naive Bayes with Laplace smoothing) against a discriminative one (Linear Regression, Ridge and LASSO, classified at a 0.5 threshold) on one shared stratified train/test split.

- **Best models:** Linear Regression and Ridge reach 0.838 test accuracy (F1 0.78, ROC-AUC 0.87). LASSO reaches 0.799 and Naive Bayes 0.732 (ROC-AUC 0.815).
- **Smoothing:** On the full training data, Laplace smoothing with $\alpha \leq 1$ does not change any test prediction (no feature/class count is zero). In a small-sample stress test, the unsmoothed model collapses to or below the 61.5% majority baseline while $\alpha = 1$ stays near 0.70-0.74.
- **Why Naive Bayes is weaker:** It over-predicts survival (31 false alarms v/s. 13 for Linear Regression) and produces overconfident probabilities, consistent with overlapping features violating its independence assumption.
- **Caution:** With only 179 test passengers, accuracy has a 95% margin of about ± 5 to ± 6.5 points, so small gaps (for example LASSO vs. Ridge) should not be over-interpreted.

1. Introduction

1.1 Problem definition

The task is binary classification: given a passenger's class, sex, age, family aboard, fare, cabin and port of embarkation, predict whether they survived the sinking of the Titanic (**Survived** = 1) or not (0). The assignment is about building a complete, defensible workflow: cleaning, feature engineering, two contrasting model families, evaluation and honest interpretation.

1.2 Dataset

We use the Kaggle Titanic **train.csv** file: 891 passengers and 12 columns. It is the only file with the **Survived** label, so we create our own stratified 80/20 split from it (712 training and 179 test rows, random seed 42). Kaggle's test.csv has no labels and therefore cannot provide accuracy or a confusion matrix; it is not used.

Column	Type	Meaning	Missing
Survived	binary target	1 = survived, 0 = died	0
Pclass	ordinal 1/2/3	Ticket class (1 = highest)	0
Name, Ticket	text	Passenger name; ticket code	0
Sex	categorical	female / male	0
Age	numeric	Age in years	177 (19.87%)
SibSp, Parch	integer	Siblings/spouses and parents/children aboard	0
Fare	numeric	Ticket price	0
Cabin	text	Cabin number	687 (77.10%)
Embarked	categorical	Port: C, Q or S	2 (0.22%)
PassengerId	identifier	Row id	0

The target is moderately imbalanced: 549 passengers died (61.62%) and 342 survived (38.38%). A classifier that always predicts "died" therefore scores 61.5% on our test split, which is the baseline every model must beat. This imbalance is also why we stratify the split and report precision, recall and F1 in addition to accuracy.

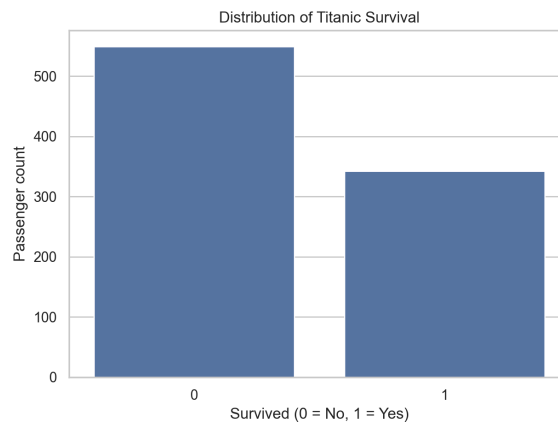


Figure 1. Distribution of the target variable in train.csv.

1.3 Workflow and leakage control

The pipeline is raw data → cleaning → feature engineering → model training → evaluation. Its central design rule is **no data leakage**: any step that learns a number from data (a median, a mean and standard deviation, a category list, quantile edges) is fitted on the training partition only and then applied unchanged to the test partition. The target never appears in the feature matrices, and one split is shared by every model so that the comparison is fair.

2. Data Cleaning

Cleaning is organized in three stages: an **audit** of what is actually wrong with the data, **deterministic fixes** that learn nothing from other rows and **statistical imputation** fitted on the training split only.

2.1 Data-quality audit

Check	Result	Action
Missing values	Cabin 687 (77.10%), Age 177 (19.87%), Embarked 2 (0.22%)	Treated individually (see 2.2)
Exact duplicate rows	0	None removed
Categorical domains	Sex = {female, male}; Embarked = {C, Q, S}; no typos	Whitespace/case normalized defensively
Invalid numeric values	Negative Age/Fare/SibSp/Parch: 0; Pclass outside 1-3: 0; Survived outside 0/1: 0	No correction needed
Zero fares	15 passengers (5 first, 6 second, 4 third class)	Left unchanged (not demonstrably wrong)

The audit found no impossible numeric values and no inconsistent categories, so there was no noisy data that required “repair.” We report this honestly rather than inventing corrections. Known quirks remain as-is: half-year ages such as 24.5 (likely estimates), infant ages such as 0.42 and extreme fares (max 512.33), which the later $\log_{1p}(\text{Fare})$ transform compresses without treating them as errors. A zero fare is not impossible (it may reflect a complimentary or unrecorded ticket) and cannot be verified, so those 15 values are kept; the heavy right skew of Fare is handled by that same transformation in feature engineering.

2.2 Missing-value decisions: Impute, Drop or Flag

Variable	Missing	Decision	Justification
Age	19.87%	Impute with the median of the training split (28.5 years)	Dropping would discard about 1 in 5 passengers. The median is robust to the right-skewed age distribution.
Embarked	0.22%	Impute with the most frequent category of the training split (S)	Only 2 rows in a categorical variable; the mode is the least intrusive fill.
Cabin	77.10%	Do not impute; create the binary flag CabinKnown	Too sparse to reconstruct without inventing values. Whether a cabin was recorded is itself informative.

CabinKnown supports its own inclusion: On the training split, 70.0% of the 160 passengers with a recorded cabin survived versus 29.2% of the 552 without one. We note that this flag overlaps with passenger class, which we return to in the Discussion.

2.3 Leakage-safe procedure

Deterministic steps (trimming and case-normalising Sex, Embarked and Cabin; creating CabinKnown) use only a passenger's own row and are applied before the split. The data is then split (stratified on Survived: 38.34% survivors in train, 38.55% in test). The median and mode are learned from the 712 training rows only and used to transform both partitions, so no test passenger influenced any imputed value. Afterwards, Age and Embarked have zero missing values in both partitions, no rows are dropped and the target is absent from the features.

2.4 Before and after

Example	Before	After
Training rows with missing Age	137 missing (test: 40)	0 missing; each filled with 28.5
Passengers 62 and 830 (Embarked missing)	missing	S (both in the training split)
Training Age mean / standard deviation	29.81 / 14.49 (575 observed)	29.56 / 13.03 (712 rows)
Cabin	string or missing (77.10% missing)	CabinKnown = 1 for 204 passengers, 0 for 687

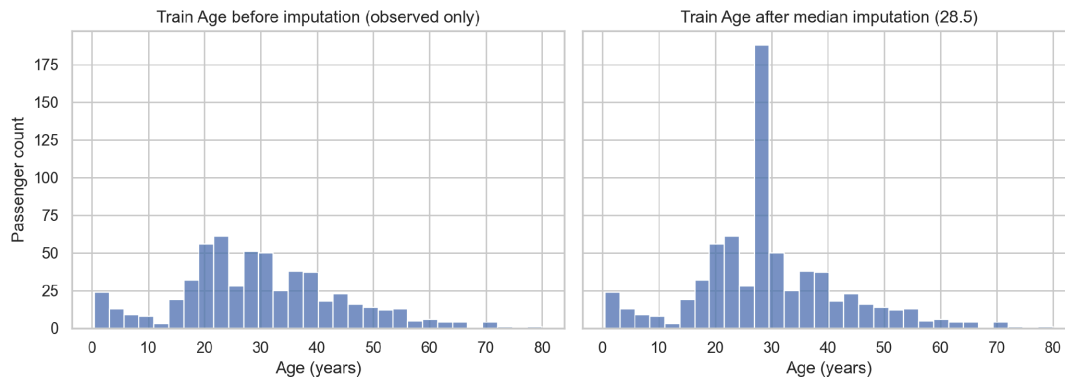


Figure 2. Training-set Age before (observed values only) and after median imputation. The spike at 28.5 and the smaller standard deviation are the known side effects of median imputation.

Median imputation keeps all passengers but concentrates 137 values at one number, which narrows the spread of Age (14.49 to 13.03). We accept this trade-off and mitigate it in Step 2 with a flag that records which ages were imputed.

3. Feature Engineering

Step 2 turns the cleaned data into numeric matrices. A feature is kept only if it has a meaningful interpretation, uses no target information, suits the model and can be explained.

3.1 Constructed features

Feature	Definition	Why
Title	Word between the comma and the period in Name; Mlle/Ms → Miss, Mme → Mrs; anything but Mr, Miss, Mrs, Master → Rare	Encodes sex, marital status and age (Master = young boy) in one compact variable; rare titles are pooled because each has only a handful of passengers
FamilySize, IsAlone	SibSp + Parch + 1; FamilySize = 1	Traveling with family changes survival; replaces SibSp/Parch
Age_missing	1 if Age was missing before imputation	Preserves the information erased by median imputation
LogFare	$\log(1 + \text{Fare})$	Fare is heavily right-skewed; log1p stays defined at Fare = 0
AgeGroup (Naive Bayes only)	child ≤ 12 , teen ≤ 18 , young adult ≤ 35 , adult ≤ 60 , senior > 60	Gives the Bernoulli model binary life-stage events; edges are fixed, not fitted

Each feature was checked against the target on the **training** split only. Title separates passengers more finely than Sex alone (Figure 3): Mrs 78.5% and Miss 70.8% survived versus Mr 15.3%, while Master (young boys) sits at 54.8%. Passengers with a missing age survived at 29.2% versus 40.5% for those with a recorded age, and passengers traveling alone at 30.0% versus 51.4% with family. These are associations, not causal claims.

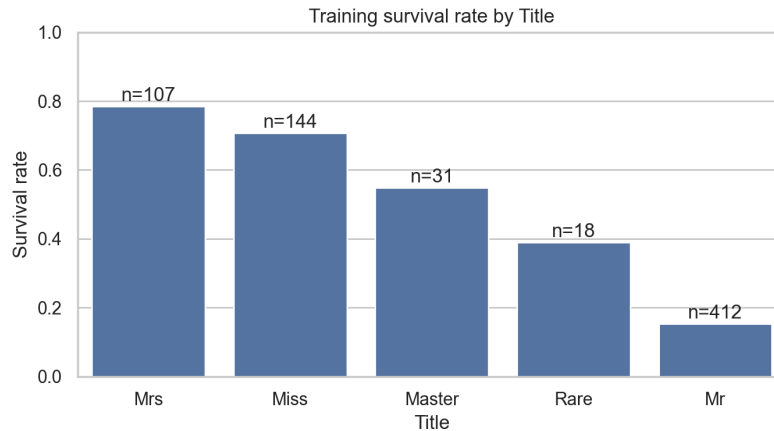


Figure 3. Training survival rate by Title (n = passengers per title).

3.2 Transformations

- **Log-scaling:** The skewness of Fare falls from 4.65 to 0.31 after $\log(1 + \text{Fare})$ (Figure 4).
- **Standardization (Linear view):** Age, LogFare and FamilySize are scaled to mean 0 and standard deviation 1 using training statistics, so coefficients are comparable and the Ridge/LASSO penalties treat features equally. On the test split the same transform gives means near 0 (-0.04 to 0.07) and standard deviations 0.95-1.08, the expected signature of a scaler fitted on training data only.
- **Binning (Naive Bayes view):** LogFare is cut into four quantile bands whose edges are learned on the training split (fare edges of about 7.90, 14.45 and 31.00); Age uses the fixed AgeGroup bins.

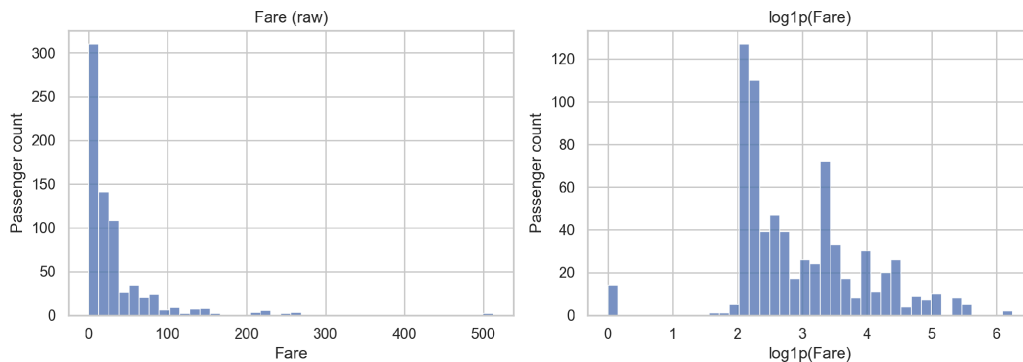


Figure 4. Fare before and after the $\log(1 + x)$ transformation (training split).

3.3 Encoding and the two feature views

Because Naive Bayes and Linear Regression make different assumptions, the same split is represented twice. **Pclass** is treated as categorical, not as a quantity.

View	Used by	Contents	Columns
Linear	Linear Regression, Ridge, LASSO	Standardized Age, LogFare, FamilySize; one-hot Pclass, Sex, Embarked, Title with explicit reference levels (class 1, female, port S, title Mr); IsAlone, CabinKnown, Age_missing	15
Naive Bayes	BernoulliNB	Only 0/1 columns: one-hot Pclass, Sex, Embarked, Title and AgeGroup; four fare-quantile bands; IsAlone, CabinKnown, Age_missing	24

Excluded: PassengerId (an identifier), raw Name (every value unique; Title is extracted instead), Ticket (571 distinct values in 712 rows), raw Cabin (replaced by CabinKnown), and SibSp/Parch/Fare (replaced by the engineered versions). Dropping a reference level per categorical variable and removing SibSp/Parch (which

sum exactly to FamilySize minus one) removes exact collinearity: the linear design matrix has full column rank. Both views have 712 training and 179 test rows with identical columns and no missing values.

4. Model Comparison

4.1 Training setup

Aspect	Setting
Split	One stratified 80/20 split of train.csv (712 / 179), seed 42, shared by all models
Generative model	BernoulliNB on the 24 binary columns; Laplace/add-alpha smoothing with alpha in {0 (none), 0.01, 1, 10, 100}
Discriminative models	LinearRegression, Ridge (alpha = 1) and LASSO (alpha = 0.01) on the 15-column linear view; class = 1 if the predicted score ≥ 0.5
Hyperparameters	Fixed in advance; none selected from test results
Metrics	Accuracy and confusion matrix (required); precision, recall, F1; ROC-AUC ; positive class = survived

Bernoulli Naive Bayes estimates, for each class, the probability that each binary feature is 1, as $(\text{count} + \alpha) / (\text{class rows} + 2 \alpha)$, plus the class prior and combines them assuming features are independent given the class. With $\alpha = 0$, a feature value never seen in a class has probability zero and can veto that class; $\alpha = 1$ is classical Laplace smoothing; large α pulls every probability toward 0.5. **Linear Regression** fits the 0/1 target by least squares; Ridge adds an L2 penalty and LASSO an L1 penalty. Its continuous score is not a probability; the assignment's 0.5 threshold turns it into a class.

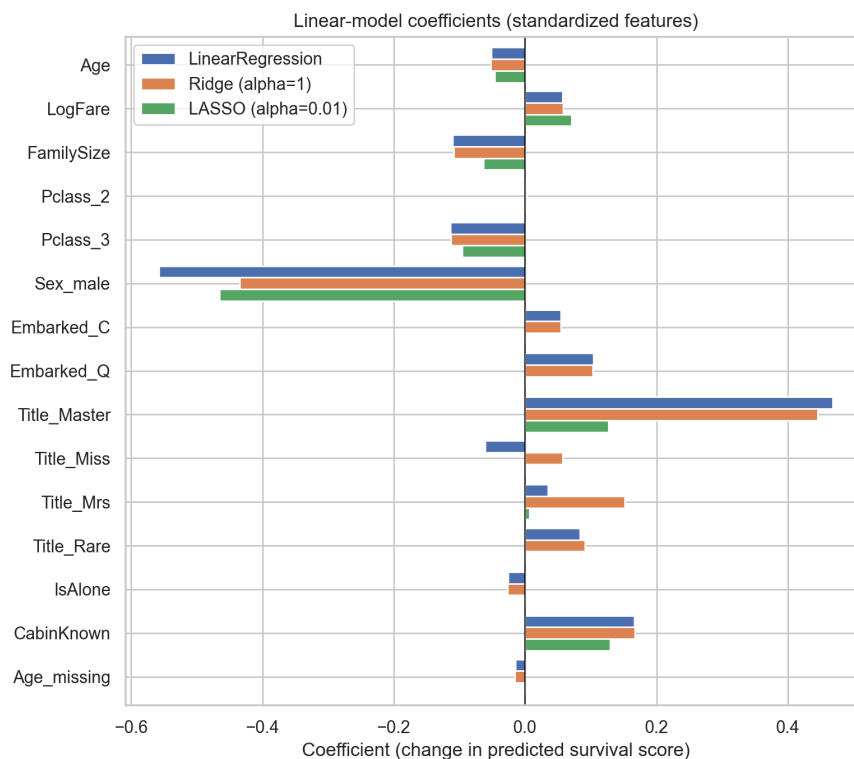


Figure 5. Coefficients of the three linear models (standardized features; dummies relative to the reference level).

In all three linear models being male lowers the survival score most (about -0.56 unpenalized) and the Master title raises it (+0.47); third class and larger families lower it, a known cabin raises it. Ridge redistributes weight between the collinear Sex and Title variables and LASSO at $\alpha = 0.01$ sets 7 of 15 coefficients exactly to zero (Pclass_2, Embarked_C, Embarked_Q, Title_Miss, Title_Rare, IsAlone, Age_missing).

4.2 Evaluation results

Model	Accuracy (95% half-width)	Precision	Recall	F1	ROC-AUC
BernoulliNB, alpha = 0 / 0.01 / 1	0.732 (± 0.065)	0.627	0.754	0.684	0.815
BernoulliNB, alpha = 10	0.726 (± 0.065)	0.625	0.725	0.671	0.821
BernoulliNB, alpha = 100	0.771 (± 0.062)	0.759	0.594	0.667	0.819
Linear Regression	0.838 (± 0.054)	0.803	0.768	0.785	0.868
Ridge (alpha = 1)	0.838 (± 0.054)	0.812	0.754	0.782	0.867
LASSO (alpha = 0.01)	0.799 (± 0.059)	0.770	0.681	0.723	0.849
Majority-class baseline	0.615	-	-	-	0.500

Table 1. Test-set metrics (179 passengers; survived = positive class). Alpha = 0, 0.01 and 1 for Naive Bayes give identical predictions.

Linear models: confusion matrices (test set, threshold 0.5)

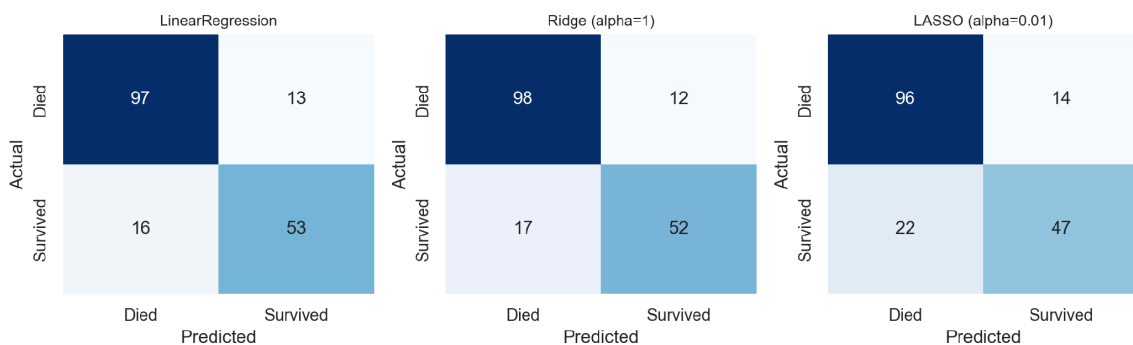


Figure 6. Confusion matrices of the linear models on the test set (threshold 0.5).

Naive Bayes: confusion matrices (test set)

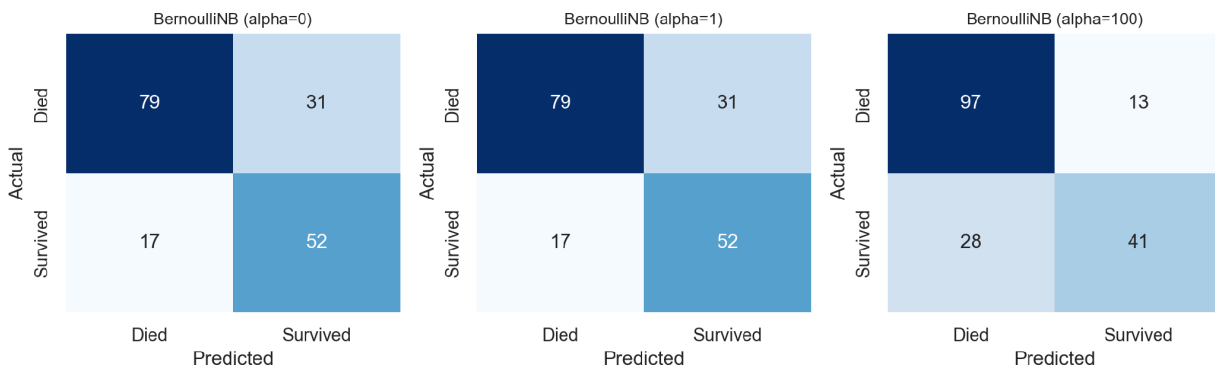


Figure 7. Naive Bayes confusion matrices without smoothing (alpha = 0), with Laplace smoothing (alpha = 1) and with heavy smoothing (alpha = 100). Alpha = 0.01 matches alpha = 0 and alpha = 1; alpha = 10 is in Table 1.

Every model beats the 61.5% baseline. The linear models lead on accuracy, F1 and AUC. The error profiles differ more than the accuracies suggest: Naive Bayes ($\alpha \leq 1$) finds nearly as many survivors as Linear Regression (52 vs. 53) but raises 31 false alarms versus 13 and it labels 46.4% of test passengers as survivors when only 38.5% survived. Heavy smoothing ($\alpha = 100$) removes the false alarms (13) but misses more survivors (28, recall 0.59): it moves the operating point rather than improving the model.

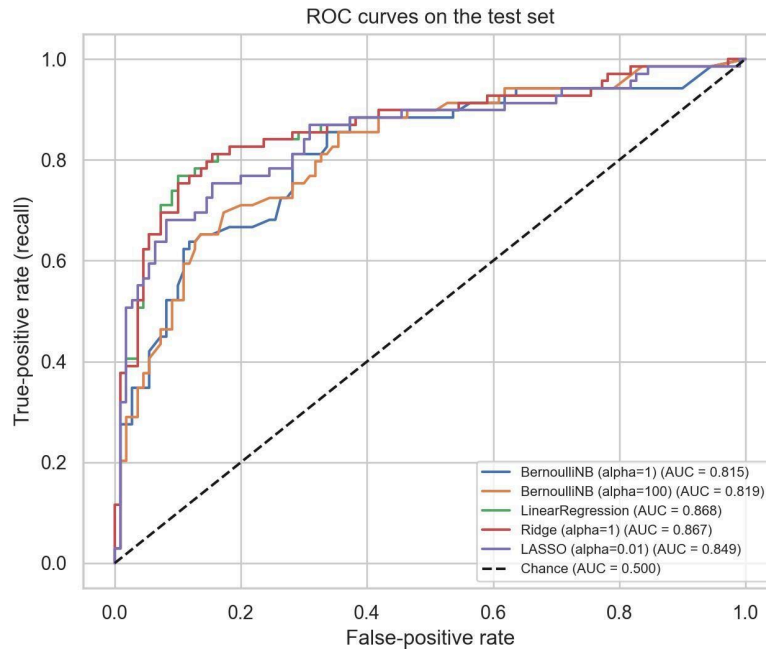


Figure 8. ROC curves on the test set. Naive Bayes with $\alpha = 0$, 0.01 and 10 are omitted because their curves nearly coincide with $\alpha = 1$.

The ranking gap (AUC 0.815 vs. 0.868) is smaller than the accuracy gap (0.732 vs. 0.838): Naive Bayes ranks passengers reasonably well, but its 0.5 cut-off is badly placed because its probabilities are overconfident. With 179 test rows, the 10.6-point accuracy gap between Naive Bayes and Linear Regression exceeds either 95% margin, although the intervals overlap slightly, so the evidence favors the linear models without being conclusive. The 4-point LASSO-Ridge gap is within noise.

4.3 Smoothing versus no smoothing

On the full training data, smoothing barely matters. The rarest feature/class combination (senior passengers among survivors) still has 3 training rows, so no probability estimate is ever zero, even without smoothing. $\alpha = 0.01$ and $\alpha = 1$ give exactly the same 179 test predictions as $\alpha = 0$ and the largest change in any predicted probability is 0.039.

alpha	Test accuracy	Predictions changed vs. alpha = 0	Max change in P(survived)	Confident predictions ($P < 0.05$ or > 0.95)
0 (none)	0.732	0	0.000	62.6%
0.01	0.732	0	0.000	62.6%
1 (Laplace)	0.732	0	0.039	62.0%
10	0.726	3	0.136	59.2%
100	0.771	29	0.456	49.7%

Table 2. Effect of the smoothing strength on the full-data Naive Bayes model.

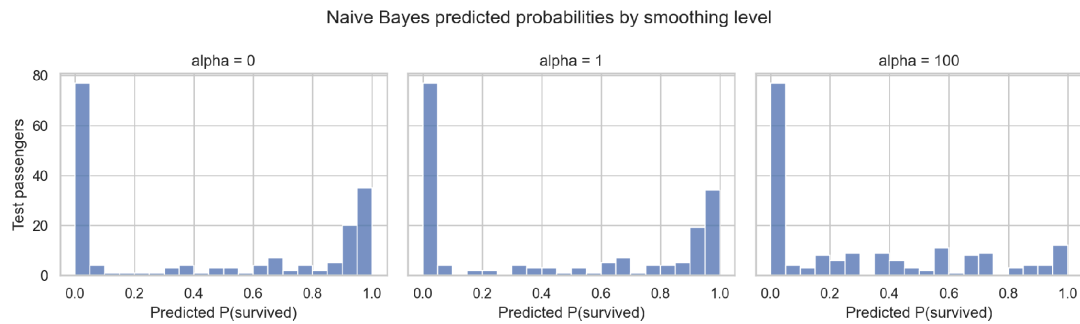


Figure 9. Predicted $P(\text{survived})$ on the test set for $\alpha = 0, 1$ and 100 .

Even unsmoothed, the model is highly polarized: about 62% of its predictions have $P(\text{survived})$ below 0.05 or above 0.95, although it is right only 73% of the time. Heavy smoothing ($\alpha = 100$) pulls probabilities toward the prior (confident predictions fall to about 50%) and shifts the errors from false alarms to missed survivors. Its higher accuracy is not a reason to choose it, since that would amount to tuning on the test set; its AUC is almost unchanged (0.819 vs. 0.815).

Small-sample stress test: Smoothing exists to protect against feature values unseen in a class. Because our training set is large enough that this never occurs, we repeat the experiment on random training subsets (100 subsets per size, fixed seed, the same subsets for every α) and score each model on the full test set. This experiment illustrates the mechanism; nothing is selected from it.

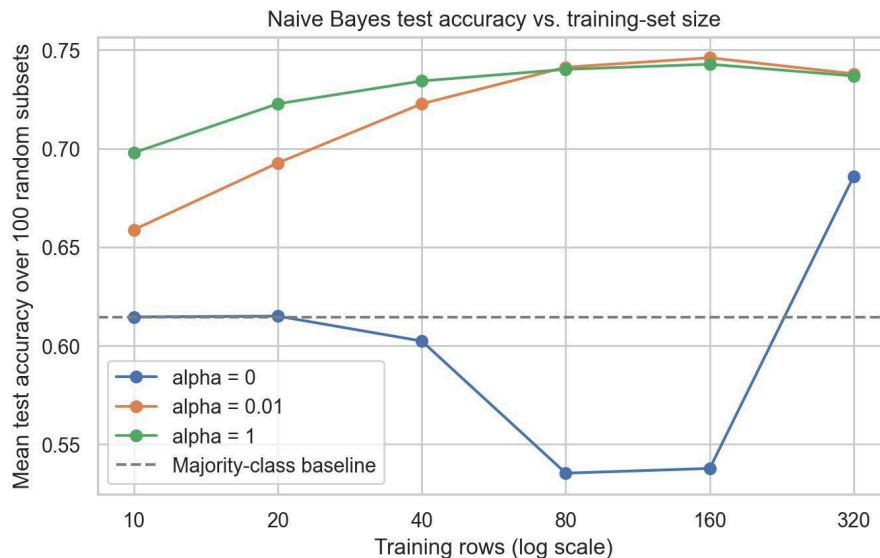


Figure 10. Naive Bayes test accuracy versus training-set size for three smoothing levels.

Training rows	$\alpha = 0$	$\alpha = 0.01$	$\alpha = 1$	Subsets with a zero-probability estimate ($\alpha = 0$)
10	0.615	0.659	0.698	100%
20	0.615	0.693	0.723	100%
40	0.602	0.723	0.734	100%
80	0.535	0.741	0.740	93%
160	0.538	0.746	0.743	64%
320	0.686	0.738	0.737	15%

Table 3. Mean test accuracy over 100 random training subsets.

Without smoothing, small samples break the model: from 10 to 160 training rows its mean accuracy (0.54-0.62) is at or below the 61.5% majority baseline, because a single zero-probability feature gives a class a score of exactly zero whatever the other features say. Alpha = 1 gives 0.70 from only 10 rows and about 0.73 from 40. Alpha = 0.01 removes the exact zeros and recovers most of the accuracy but trails alpha = 1 on the smallest samples, because a probability of 0.01 is still extreme; from about 80 rows the two are equivalent. The effect fades as data grow, which is why smoothing makes no difference on the full 712 rows.

5. Discussion

5.1 Interpretation of the results

- **Discriminative beat generative here.** Linear Regression and Ridge reached 0.838 accuracy against 0.732 for Naive Bayes. A plausible explanation is the independence assumption: Sex and Title overlap heavily and so do Pclass, the fare bands and CabinKnown, so Naive Bayes counts the same evidence several times. This shows up as overconfident probabilities and excess false alarms. This is an interpretation consistent with the evidence (probability histogram, confusion matrices, AUC gap), not something we prove with an ablation.
- **Regularization helped little.** Ridge matched plain Linear Regression because, with a dozen standardized features and 712 rows, overfitting is mild (train and test accuracy are close for every model). LASSO's sparser model lost recall (0.68) and accuracy (0.799), consistent with the stronger shrinkage of informative effects.
- **Features carried the signal.** Sex and the Master title dominate the coefficients; class, family size, age and a known cabin contribute smaller, sensible effects. Missing age and being alone were associated with lower survival.
- **Smoothing is a safeguard whose value depends on data size.** It was essential for small samples, irrelevant at mild strength on the full data, and distorting when too strong.

5.2 Strengths

- A leakage-safe pipeline in which every learned parameter comes from the training split, verified by explicit checks.
- A fair comparison: one stratified split, fixed hyperparameters, identical metrics for all eight model variants.
- Decisions justified by evidence from the actual data (audit, missingness, training-set survival rates) rather than convention.
- Honest handling of weak points: a clean audit is reported as clean and the smoothing result is explained rather than forced.

5.3 Limitations

- **Small test set and one split.** 179 test passengers give accuracy margins of about ± 5 to ± 6.5 points; cross-validation would give more reliable estimates but was outside the assignment's scope.
- **Linear Regression is not a probability model.** Its scores ranged from -0.49 to 1.14, with 5% outside [0, 1]; the 0.5 threshold is well defined but the scores are not calibrated.
- **Independence violated for Naive Bayes** by overlapping features, a consequence of our feature design.
- **Imputation side effects.** Median imputation shrinks the spread of Age, and Age_missing only partly compensates. Group-wise imputation (for example by title) might be more accurate but adds complexity.
- **Overlapping signals.** CabinKnown overlaps with Pclass, and Title with Sex, so individual coefficients should be read jointly. AgeGroup edges are conventional rather than optimized.

- **Generalization.** Titanic is a small teaching dataset; the results say little about production performance. A deployed system would favor logistic regression or tree models, cross-validation and calibration.

6. AI Tool Usage Disclosure

6.1 AI tools used

- ChatGPT (OpenAI)
- Cursor
- CodeRabbit

6.2 What the AI tools contributed

- Generated and refined starter and final Python code for:
 - Data loading
 - Data cleaning
 - Feature engineering
 - Model training
 - Model evaluation
 - Visualizations
- Assisted with implementing the leakage-safe train/test split and preprocessing workflow.
- Assisted with implementing and debugging:
 - Naive Bayes
 - Laplace/add- α smoothing experiments
 - Linear Regression
 - Ridge and LASSO
 - Evaluation metrics and visualizations
- Provided conceptual explanations of:
 - Missing-value imputation
 - Data leakage
 - Categorical encoding
 - Laplace smoothing
 - Regularization
 - ROC/AUC and other evaluation metrics
- Assisted with debugging and consistency checks to identify potential implementation or interpretation issues.
- Assisted with drafting and refining portions of the report structure, tables, figure descriptions and written analysis.
- CodeRabbit was used for limited automated PR/code reviews to identify potential issues and inconsistencies during development.

6.3 Self contribution

- Selected the Titanic Survival Dataset and established the overall workflow according to the assignment requirements.
- Defined the methodological constraints used throughout the analysis, including:
 - Avoiding data leakage
 - Using a consistent train/test split for model comparison
 - Following the requirements specified in the assignment
- Reviewed the proposed preprocessing and modeling approaches and made/approved the final methodological decisions, including:
 - Missing-value treatment
 - Cabin information representation
 - Feature construction
 - Model configurations
- Ran the notebook in **Google Colab** and inspected the resulting outputs, tables and visualizations.
- Verified that the implemented workflow and results were consistent with the actual dataset and written interpretations.
- Evaluated and validated the analysis, model comparisons, visualizations and conclusions.
- Reviewed and refined the AI-assisted code and explanations throughout the development process.

7. Additional Resource

Github Link: <https://github.com/2004Luci/AML>